

Drone Garage Updates

11/21/25

Getting ESP support on Arduino

- In IDE
 - File -> Preferences -> “Additional Board Manager URLs”
 - Paste http://arduino.esp8266.com/stable/package_esp8266com_index.json
 - In Board Manager find “esp8266”, install
- Need Silabs drive
<https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>
- Note: When you want to serially monitor an ESP, if it is telling you it's not connected, try closing serial monitor, press RST on ESP, then reopen serial monitor

WiFi Rework

- To my understanding, the mesh functionality remains mostly the same
- What we have to change is the gateway connection being from qgc to the esp
 - Hold on doesnt that defeat the purpose of the pi esp when we're using it for other things?
Forgot what they said in the meeting
- This new connection will be via 802.11S
 - Oh and the mesh might actually have to be reworked because painlessmesh uses the whole thing
 - Hold on he mightve said that we should use 802.11S fo the mesh so that we can mesh it while having the gateway connection or smth? ask Ishan more abt this later

MavLink over Reticulum

- U need Rust for this btw
- Just in case i cant set up the entire flow im going to set up what i can on the Canakit Pi
 - `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh → source "$HOME/.cargo/env" → Git clone https://github.com/BeechatNetworkSystemsLtd/rns-mavlink-rs → cd rns-mavlink-rs → cargo build`
 - Configure connection
 - Configure in Gc.toml and Fc.toml appropriately (the former is for the system with ground control (our QGC) and the latter is for the system with the flight controller (the pi))
 - For now ill configure Fc.toml
 - So we need...
 - Serial_port = the port where the flight controller is connected (ez)
 - Serial_baud = baud rate of the port (also ez)
 - Gc_destination = reticulum address hash of the gc node (huh???)
- Reticulum
 - Gonna duplicate the venv we got going on rn
 - <https://stackoverflow.com/questions/7438681/how-to-duplicate-virtualenv>
 - `Python3 -m venv ~/mavenv_r → source ~/mavenv/bin/activate → pip freeze > requirements.txt → deactivate → source ~/mavenv_r/bin/activate → pip install -r requirements.txt`
 - **Continue to next slide...**
- Resources
 - <https://rust-lang.org/tools/install/> (Rust stuff)
 - <https://reticulum.network/manual/gettingstartedfast.html#develop-a-program-with-reticulum> (Reticulum stuff)
 - <https://beechat.network/2025/10/24/beechat-network-systems-unveils-mavlink-over-reticulum-toward-zero-trust-drone-command-control/> (The mavlink over reticulum guys)
 - <https://github.com/BeechatNetworkSystemsLtd/rns-mavlink-rs/tree/master> (their git)
 -

MavLink Over Reticulum 2

- Reticulum
 - In mavenv_r venv:
 - Pip install rns → [ended here]
- Resources

No Longer Working with MavLink Over Reticulum

- Sticking to ESPs due to time constraints, unnecessary implementation overhead, and that they're lightweight enough that they're no big deal to have with the Pis
- Next:
 - Confirming the full mesh dataflow
 - Extend connection range for gateway drone using usb adapter for wifi

Also Finish the Current Functionality

- Make sure the MAVLink data when sent thru the mesh is actually readable on the gateway and on the other ESP Pi setups
- Recreate what we did with the original Pi on other Pi ESP combinations to test this

Pi 4 installation

- Sudo raspi-config -> interface options -> login shell over serial No -> enable serial hardware Yes -> reboot
- Install mavlink-router -> git clone <https://github.com/mavlink-router/mavlink-router>.
git -> git submodule update --init --recursive
 - Previous link in other slides did not work, could not find repository
 - This repository needs recursive installation of another library
- cd mavlink-router -> sudo apt install -y ninja-build -> meson setup -Dsystemdsystemunitdir=/usr/lib/systemd/system build . -> ninja -C build -> sudo ninja -C build install

- Connect to drone -> sudo mkdir -p /etc/mavlink-router -> sudo nano /etc/mavlink-router/main.conf
- Copy into config:

```
[General]
TcpServerPort=5760
UdpPort=14550
ReportStats=false
```

```
[UartEndpoint pixhawk]
Device=/dev/serial/by-id/usb-Holybro_Pixhawk6C_4C0027000D51333130363431-if00 (run ls -l /dev/serial/by-id to see device)
Baud=115200
FlowControl=false
```

```
[UartEndpoint esp8266]
Device=/dev/ttyAMA0
Baud=57600
FlowControl=false
```

Pi 4 continued

- `sudo nano`
`/etc/systemd/system/mavlink-routerd.service`
- Copy into service:

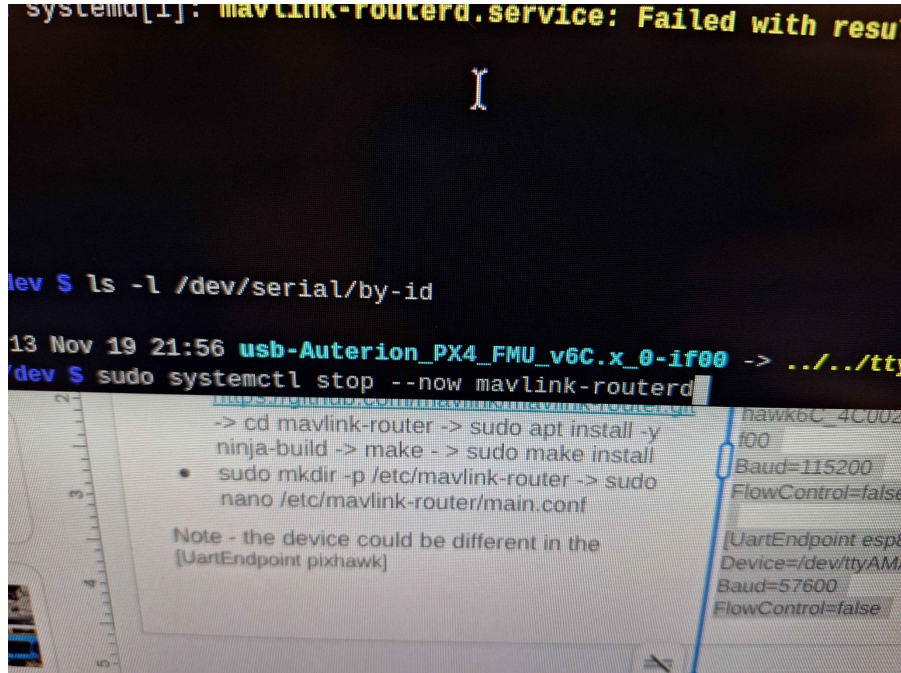
```
[Unit]
Description=MAVLink Router
After=network-online.target
Wants=network-online.target
```

```
[Service]
Type=simple
ExecStart=/usr/bin/mavlink-routerd -c
/etc/mavlink-router/main.conf
Restart=on-failure
RestartSec=2
```

```
[Install]
WantedBy=multi-user.target
```

- `sudo systemctl`
`daemon-reload`
- `sudo systemctl enable`
`--now mavlink-routerd`
- `sudo systemctl status`
`mavlink-routerd`

Pi 4 issue

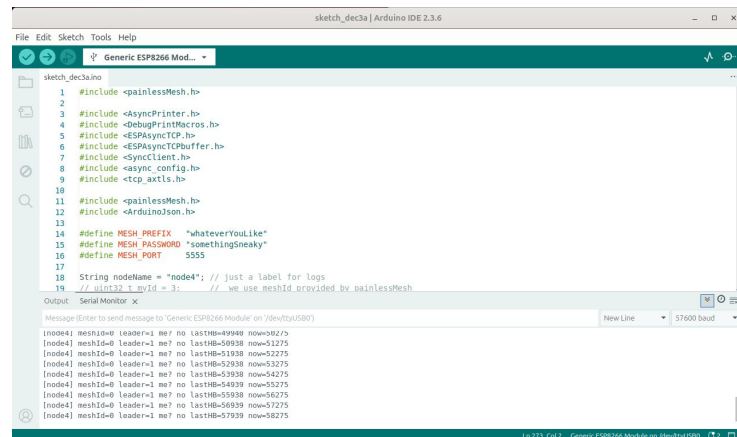


- ESP not recognized, only displays the drone
- Not entirely sure what could be the cause (ESP itself, pi, wiring, etc)
- Needs to be looked at a bit more
- **Now resolved**

Confirming Dataflow

ESP data detection

- Arduino IDE serial monitor of ESP not detecting drone data from leader
- Next Steps:
 - Boot both pi setups (no password or anything required; refer to images in the next slide) → select one of the drones and connect it serially (usb to usb c) with a selected pi (this will be the leader) (**refer to 11/14 slidedeck for qgc to pi connection**) → test various versions of mesh-network/mesh-gateway/mesh-gateway.ino from the github on the esp8266 (refer to one of the previous slide decks for uploading code to esp8266) **to see which version actually forwards leader drone data to the rest of the mesh** (if none work try to mess with the files and develop functionality for it; one of them should work but also like we're tryna make sure real drone data can be sent (one of them was able to send fake data throughout the mesh)) → **whatever happens just document it in the slides** → if u can, also see if u can work on particular leadership election cases (such as when the leader goes down and qgc has to connect to the newly elected leader and stuff like that (refer to the last 11/14 slide))

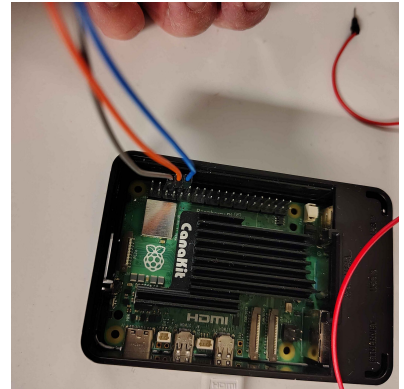
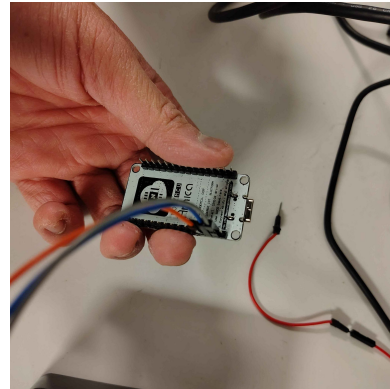
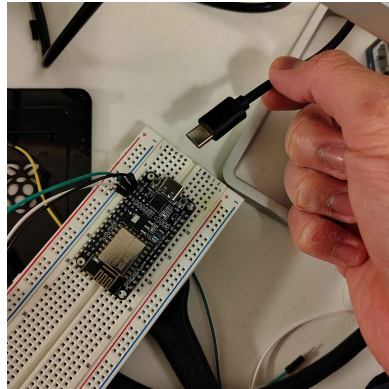
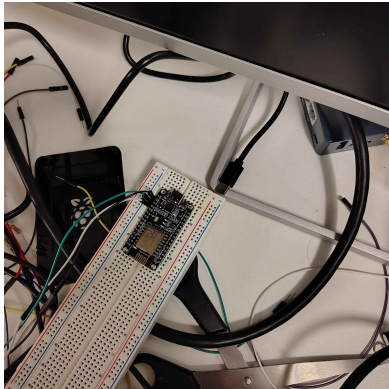
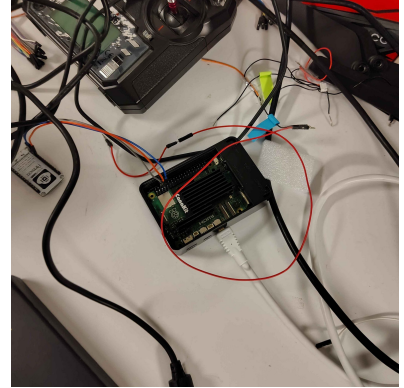
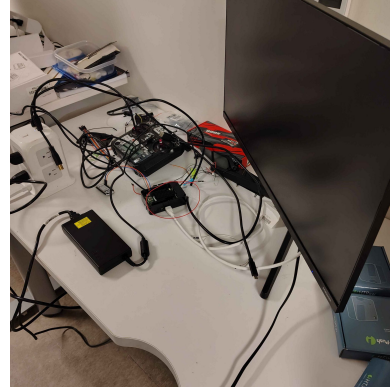
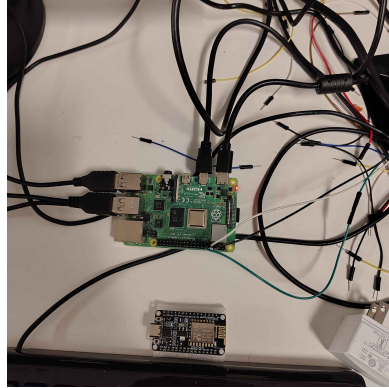
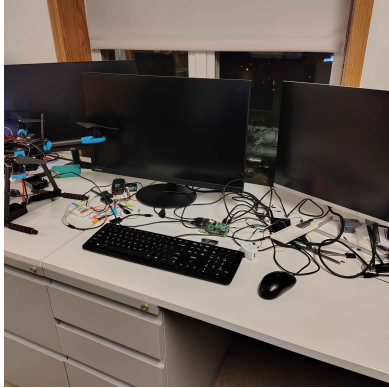


The screenshot shows the Arduino IDE interface. The sketch editor displays the code for 'sketch_dec3a.ino', which includes various libraries and defines mesh-related parameters. The serial monitor shows the output of the code, displaying a series of messages from the leader node to the mesh nodes, indicating the mesh ID and the last time the mesh was updated.

```
sketch_dec3a.ino
1 #include <painlessMesh.h>
2
3 #include <AsyncPrinter.h>
4 #include <DebugPrintMacros.h>
5 #include <ESPAsyncTCP.h>
6 #include <ESPAsyncTCPpuffer.h>
7 #include <SyncClient.h>
8 #include <axtls.h>
9 #include <tcp_axtls.h>
10
11 #include <painlessMesh.h>
12 #include <ArduinoJson.h>
13
14 #define MESH_PREFIX "whateverYouLike"
15 #define MESH_PASSWORD "somethingSneaky"
16 #define MESH_PORT 5555
17
18 String nodeName = "node4"; // just a label for logs
19 // uikit32 t gvid = 3; // we use meshid provided by painlessMesh

Output: Serial Monitor x
Message (Enter to send message to 'Generic ESP8266 Module' on '/dev/ttyUSB0')
[Node4] meshId=0 leader=1 me7 no lastHb=49948 now=50275
[Node4] meshId=0 leader=1 me7 no lastHb=50938 now=51275
[Node4] meshId=0 leader=1 me7 no lastHb=51938 now=52275
[Node4] meshId=0 leader=1 me7 no lastHb=52938 now=53275
[Node4] meshId=0 leader=1 me7 no lastHb=53938 now=54275
[Node4] meshId=0 leader=1 me7 no lastHb=54939 now=55275
[Node4] meshId=0 leader=1 me7 no lastHb=55938 now=56275
[Node4] meshId=0 leader=1 me7 no lastHb=56939 now=57275
[Node4] meshId=0 leader=1 me7 no lastHb=57939 now=58275
```

Wiring, Setups, etc (If what you're looking for isn't here look at previous slide decks)



Crazyflie?

- Idk its actually smaller than a pi so i think its not the move